



EMBEDDED SOFTWARE ENGINEERING

Session 7 – Lab: the Runner

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today: build a game

- A complete lab, on your own: a *runner* (think Chrome's offline dinosaur) running on the board, displayed on the OLED
- Everything seen in the course is needed: C objects, events, timers, state machines
- A lab in 5 steps, each one ends with something you can **see on the screen** – show it to me before moving on
- Not everybody will finish in 2 hours, and that's fine: the last steps and the bonuses make a good evening project

Today

- 1 Starting Point
- 2 Step 0: Read the Code
- 3 Step 1: A Running Dino
- 4 Step 2: A Cactus
- 5 Step 3: Collision and Game Over
- 6 Step 4: Many Cacti, at Random
- 7 Step 5: Score and Difficulty
- 8 Going Further

What you get

- The project from session 6 (menu, LEDs, NeoPixel, time bomb), plus a fourth line in the main menu: **RUNNER**
- Already working: from the menu, RUNNER → press SW2, and a **rectangle jumps** each time you press SW2 again
- Already given: the screen's integration (menu entry, app mode), the game tick, the dino and its jump, a pseudo-random generator
- Your job: everything that turns this rectangle into a game

Same recipe as the time bomb

The runner is not a menu: like the time bomb it is a *standalone screen*, with its own state machine, that takes over the display and the buttons until SW1.

The files

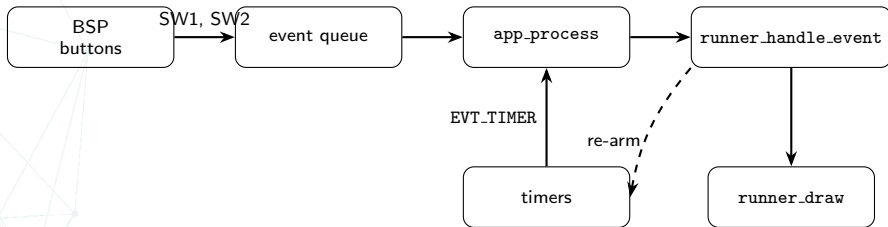
```

app/
|-- app.c                <- unchanged logic: dispatches events to the active screen
|-- runner/
|   |-- runner.c/.h      <- the game: FSM, tick, drawing          (you complete)
|   |-- dino.c/.h        <- the player: jump FSM, drawing      (step 1)
|   `-- obstacle.c/.h    <- a cactus: move, draw, collide      (steps 2-3)
`-- menu_entry/
    `-- runner_launch_entry.c <- the menu line that starts the game (given)

```

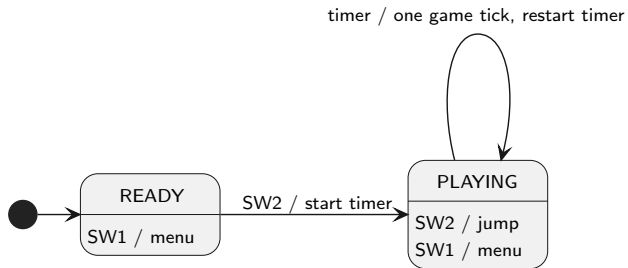
- Look for the TODO comments: each one tells you the step it belongs to
- obstacle.h is complete: it is the *contract* of the class, you only write the implementation

Where events go



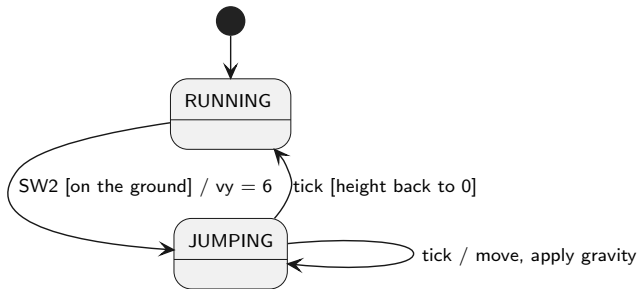
- **Nobody** reads a button or waits in the game: everything happens in response to an event, and there are only two kinds that matter – a button click, and the game timer
- After each event, `app_process` calls `runner_draw`: the screen is redrawn from the current state, from scratch

The game as a state machine



- That's what you start with: `runner_state_t` has two states
- The game timer is **periodic**, but the timer service is one-shot: see how `runner_handle_event` re-arms it

The dino: a second, smaller state machine



- Independent from the game's FSM: the game only calls `dino_jump` and `dino_update`, it never looks at the dino's internals
- `height` grows *upwards* (0 = on the ground), unlike the screen's *y* axis

Step 0 (15 min): questions

Read `runner.c` and `dino.c`, then answer, on paper:

- 1 Which event makes the game move forward, and in which function is it handled?
- 2 Why is the timer re-armed with `next_tick_ms += RUNNER_TICK_MS` rather than `now_ms + RUNNER_TICK_MS`? What would change after one minute if one tick is late?
- 3 What happens if you press SW2 twice during a jump? Which line is responsible?
- 4 Change `DINO_JUMP_SPEED` and `DINO_GRAVITY`: what happens to the jump? Compute the apex height $(v + (v - 1) + \dots + 1)$.
- 5 `dino_draw` receives a `frame` argument that nothing uses yet: what is it for?

Step 1 (15 min): make it run

- Goal: replace the rectangle in `dino_draw` by a character that **looks like it is running**
- You only have rectangles: `bsp->oled_fill_rect(bsp, x, y, w, h, on)` – `on = false` erases, handy for an eye
- Stay inside the `DINO_W×DINO_H` box starting at `(DINO_X, top)`: the collision will use it

```
void dino_draw(const dino_t *d, bsp_t *bsp, int ground_y, int frame) {  
    int top = ground_y - DINO_H - d->height;  /* y of the dino's top edge */  
    /* body, head, eye, legs... */  
}
```

- frame increases by 1 each game tick: `(frame / 2) % 2` flips every 2 ticks, which gives two poses of the legs
- In the air, `d->state == DINO_JUMPING`: legs should not move

Step 1: what you should see

- The dino stands on the ground line, its legs alternate while it runs, and freeze when it jumps
- **Be creative:** a hat, a tail, a blinking eye, a body that squashes on landing – the best dinosaur of the room gets a round of applause
- Think about it: you choose the pose from $d \rightarrow \text{state}$. Which information from the dino's state machine did you use to draw it?

Checkpoint

Show me your dino running and jumping before moving on.

Step 2 (25 min): the obstacle class

```
typedef struct {
    bool active;
    int x;    /* left edge, in screen pixels (may be > 127: not visible yet) */
    int w;
    int h;
} obstacle_t;

void obstacle_init(obstacle_t *o);
void obstacle_spawn(obstacle_t *o, int w, int h);    /* appears on the right edge */
void obstacle_update(obstacle_t *o, int speed);    /* moves left by `speed` px */
void obstacle_draw(const obstacle_t *o, bsp_t *bsp, int ground_y);
```

- Same recipe as dino: a struct, and functions taking a pointer to it as their first argument
- obstacle.h is already written: only implement obstacle.c (the four functions above)
- active = false means *the slot is free*: no malloc here, objects will live in an array

Step 2: integrate, then see

- 1 Implement `obstacle_init/spawn/update/draw`
 - 2 Add **one** `obstacle_t` to `runner_t`, and initialize it in `runner_reset_game`
 - 3 In `runner_tick`: update it, and spawn a new one when it is not active (fixed size, for now)
 - 4 In `runner_draw`: draw it
- Use a constant speed (e.g. 3 px per tick) – the speed parameter will matter in step 5
 - **Pitfall**: when does an obstacle leave the screen – when its left edge passes $x = 0$, or its right edge? Draw it on paper
 - **Pitfall**: `runner_reset_game` must also reset the obstacle, otherwise the previous game's cactus is still there

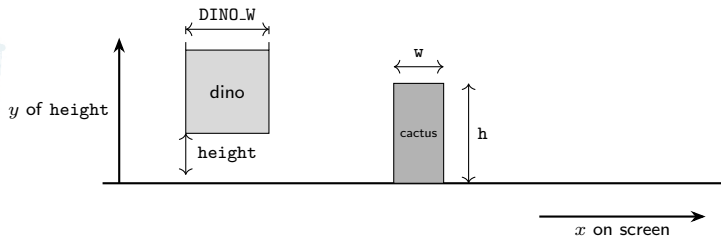
Checkpoint

A cactus crosses the screen, then another one, forever. The dino ignores it for now: it goes through.

Step 3 (20 min): update the diagram first

- Goal: touch a cactus $\Rightarrow$ the game stops, a GAME OVER screen shows up, SW2 starts a new game
- **Before** any code, on paper: redraw the game's state machine with the new state
 - Which transition leaves PLAYING, triggered by which event, with which **guard**?
 - Which **actions** run on that transition? (what must happen to the timer? to the NeoPixel?)
 - Which transition leaves GAME_OVER?
- SW1 must still go back to the menu from this new state

The collision test



- The two rectangles overlap if they overlap **both** along *x* *and* along *y*
- *x*: the intervals $[DINO_X, DINO_X + DINO_W[$ and $[x, x + w[$ intersect
- *y*: the cactus occupies heights $[0, h[$: when does the dino *not* touch it?

Step 3: to do

- 1 Implement `obstacle_hits_dino` (the guard) – it returns `false` for an inactive obstacle
 - 2 Add `RUNNER_GAME_OVER` to `runner_state_t`
 - 3 In `runner_tick` (or right after it): when the guard is true, change state, stop the timer, turn the NeoPixel red (`bsp->neopixel_set_hsv(bsp, 0, 255, 40)`)
 - 4 In `runner_handle_event`: new case, SW2 starts a new game
 - 5 In `runner_draw`: `GAME OVER` message on top of the scene
 - 6 NeoPixel back to blue (hue 240) when a game starts
- **Pitfall:** the compiler warns about a switch that doesn't handle the new enum value – read the warning, it shows you every place to update
 - **Pitfall:** test the *airborne* case too: jumping over a cactus must work!

Checkpoint

You die on the cactus, you can retry, and a well-timed jump saves you.

Step 4 (20 min): a pool of obstacles

- One cactus is boring: replace the single `obstacle_t` by an **array** of 3, a *pool*
- A free slot (`active == false`) is available to spawn a new one; if the pool is full, wait for the next tick
- Loops over the pool everywhere you used the single obstacle: update, draw, collision
- Spawn is driven by a distance counter: how many pixels until the next cactus appears?

```
r->spawn_countdown -= speed;           /* each tick, the world scrolls by `speed` px */
if (r->spawn_countdown <= 0) {
    /* find a free slot, spawn it, then pick the distance to the next one */
}
```

- Random: `runner_rand(r, n)` returns a number in $[0, n[$ – given, just use it to choose the size (e.g. 3 different widths and 3 different heights) and the gap

Step 4: how far apart?

- A jump lasts about 12 ticks. If two cacti are closer than what the dino covers during a jump, the game is **impossible**
- Use for the next gap, in pixels:

$$\text{gap} = w + 14 * \text{speed} + \text{runner_rand}(r, 50)$$

- Why does it depend on speed? Think about what happens in step 5, when the world scrolls faster while the jump keeps the same duration
- The seed of the generator is the time at which you press SW2: two games never look the same

Checkpoint

A randomly generated course of cacti, playable and fair. **You have a game.**

Step 5 (15 min): score, high score, speed

- Score: it goes up with time, e.g. `ticks / 3`; displayed on the top line (`snprintf` into a `char[]`, then `oled_draw_string`)
- High score: kept between two games – *where* do you have to store it so that `runner_reset_game` does not erase it?
- Difficulty: speed grows with the score, in steps (e.g. +1 px/tick every 100 points, between 3 and 6)
- The speed is a **guard** on the score: no new state, no new event – a condition evaluated at each tick
- Think about it: what would it take to model “level 1, 2, 3” as *states* instead? Is it worth it here?

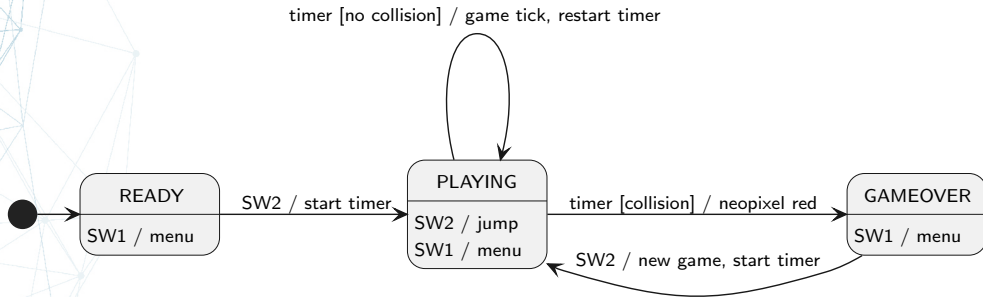
Checkpoint

Score displayed, high score kept, and the game gets noticeably harder after a few cacti.

Bonuses, from easiest to hardest

- 1 **Pause:** a click on the encoder freezes the game (new state PAUSED, the timer is stopped), another click resumes
 - 2 **Difficulty menu:** on the READY screen, turn the encoder to choose the starting speed
 - 3 **Lives:** 3 lives shown on the LEDs; hitting a cactus costs one and the dino blinks for a while (a second timer – there are 4 in the service)
 - 4 **Flying birds:** an obstacle that flies at the dino's height – which jump height saves you now?
 - 5 **Duck:** another dino state, triggered by the other button
 - 6 **Scrolling ground:** a few pixels moving to give an idea of speed
 - 7 **Night mode:** the NeoPixel shows the speed level
- For each: first the diagram (which state, which event, which guard), then the code

Target state machine (to compare with yours)



- Yours does not need to be identical – as long as every transition has an event, and every action is in the right place

Take away

- **Objects** (session 4): `dino_t` and `obstacle_t` each own their data and the functions that touch it; the game composes them and never reaches into their fields
- **Events** (session 5): no button reading, no `HAL_Delay`: the game advances only when an event arrives – the periodic tick is a re-armed one-shot timer
- **State machines** (session 6): two of them, one inside the other – the game and the dino – with guards (collision, on the ground, speed) and actions (timer, NeoPixel)
- Nothing in the game touches the HAL: the same code could run in the simulator on your laptop